

<https://youtu.be/WWhS0hRsuPs>

Jose Hernandez — MemoFunction

Hello, today we're going to go through the MemoFunction class. MemoFunction is a wrapper function class that stores results and acts like a cache.

The main idea is that it helps with expensive functions by avoiding repeated calls with the same input parameter.

Inside the class, there are two key components: an unordered map called the cache, and a queue called order. These work together to maintain the caching behavior and enforce the eviction policy.

When the function is called, it first checks the cache. If the key is found, it immediately returns the stored result. If the key is not present, then it is a cache miss.

In that case, the class checks whether the cache has reached its capacity. If it has, it removes the oldest entry using a first-in, first-out eviction policy. It finds the oldest key from the queue, removes it from the queue, and erases it from the cache.

Then the wrapped function is executed, the result is stored in the cache, the key is pushed into the queue, and the result is returned.

There are also helper functions such as Clear, which clears both the cache and the queue, and Size, which returns the current number of cached values.

Truong Phan — EventQueue

This is the EventQueue class. This class uses a heap-based data structure to determine and control which event happens next in the simulation. It is very similar to the queue and priority queue structures in the standard library.

There is a struct that represents an event. Each event holds some type of data, along with a priority value and a secondary value used for tie-breaking.

The EventQueue class contains a vector representing the heap and an insertion index that tracks when each event was added.

A custom comparator is used for tie-breaking. It first compares the priority of events, and if the priorities are equal, it returns the smaller tie-breaker value. The tie-breaker is determined by the insertion index.

The EventQueue supports pushing a new event, popping the top event, viewing the top event without removing it, checking the size, and determining whether the queue is empty.

Throughout these functions, assert statements ensure that the heap property remains valid and that programmer misuse, such as invalid event data, is caught early.

John Masterman — DataFileManager

This is the DataFileManager class. This class manages structured data files that are updated during a simulation.

The main goal is to let the user register a set of functions, execute them on demand, and write their results as a new row in a data file.

First, in the header file, the private section contains a Column struct. This struct represents one column in the file, storing both the column name and the function that produces its value.

The class contains an output stream for writing, a vector of registered columns, and a boolean flag that tracks whether the header row has been written yet.

The main supported functions are Open, Close, AddColumn, Update, and Flush.

Open prepares the file for writing. Close safely closes it. AddColumn registers a column name and a function that generates a value during updates. Update is the core method that writes a new row of data, and Flush forces output to be written immediately.

In the implementation file, the default constructor does not open a file right away, while the filename constructor opens the file immediately.

The Open function creates or truncates the file to ensure a clean schema for each run and resets the header state. The Close function safely closes the file.

AddColumn stores each column function in order, ensuring updates always produce consistent column ordering. It also uses asserts to prevent invalid column names or functions, or adding columns after writing has started.

The Update function writes the header row once using the registered column names, then calls each stored function and writes the returned values as a comma-separated row in the file.

Flush ensures buffered output is written to disk. Overall, this class is useful for simulation logging, debugging output, and collecting experiment data over time.

George Almeida — WeightedSet

My name is George Almeida, and I created the WeightedSet class for Group Eight in Company C.

At a high level, it is a templated container that stores unique elements, each with a positive weight attached. The main feature it provides beyond a regular set is weighted sampling, where an element can be chosen proportionally to its weight.

The class lives in the CSE498 namespace and takes a type parameter plus an optional comparator, similar to `std::set`.

The type aliases near the top improve readability, such as naming `double` as `weight_type` and the underlying `std::map` as `map_type`. The storage uses `std::map`, which ensures sorted unique keys.

Insert asserts that weights are strictly positive. If the element already exists, it returns `false`. Erase and find follow the same pattern, returning `false` or end on a miss rather than throwing exceptions, since missing elements are a normal outcome.

Erase also decrements the running total weight before removing the node, keeping the cumulative sum consistent.

Sampling by weight takes a cumulative weight value and walks the map linearly until the threshold is crossed. A fallback returns the last element to handle floating-point precision edge cases.

SampleRandom takes any uniform random bit generator, allowing it to work directly with our Random class. It generates a value from zero to total weight and delegates to SampleByWeight.

The private section stores the map and the running total.

Benjamin Forbes — Random

This is the Random class, a random number generator.

It uses an engine from the standard library, since it produces higher-quality randomness than simple modular arithmetic.

The class includes a flag to indicate whether the generator has been seeded, and all functions require the generator to be seeded before use.

There are two constructors: one constructs the generator with a seed immediately, and the other creates the object first and allows seeding later. This gives the user control over when randomness begins.

The Seed function seeds the generator and sets the seeded flag to true, allowing the asserts in the other functions to pass.

GetDouble returns a double within the given range. GetInt returns an integer within the given range. P performs a probability check and returns true or false based on a value between zero and one.

The Bernoulli distribution from the standard library is used for this probability function.

That covers the main functionality of the class.